

DOCUMENTACIÓN INTERNA / MANUAL DE ARQUITECTURA

Pantry Guard

Manual Técnico de Ingeniería de Software, Arquitectura Cliente-Servidor (BFF), Endpoints de Inteligencia Artificial y Despliegue en Producción

SISTEMA / VERSIÓN

Pantry Guard v1.0.0 (Full-Stack)

ENTORNO DE EJECUCIÓN

Node.js v24 LTS + Express / Vite 6

MOTOR DE IA GENERATIVA

Google Gemini (Gemini 2.5 Flash SDK)

PLATAFORMAS SOPORTADAS

Android (TWA/PWA), iOS, Web (HTTPS)

Tabla de Contenidos

ESTRUCTURA DEL MANUAL TÉCNICO

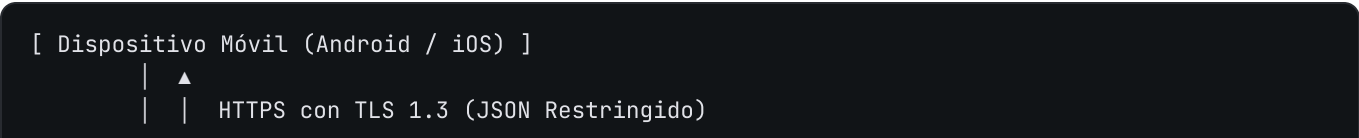
1. Resumen de Arquitectura y Patrón BFF	Pág. 2
2. Stack Tecnológico y Matriz de Dependencias	Pág. 3
3. Estructura de Directorios del Repositorio	Pág. 4
4. Modelado de Datos e Interfaces TypeScript	Pág. 5
5. Especificación de Endpoints y Servicios API	Pág. 6
6. Mecanismos de Visión Computacional y Escaneo 1D	Pág. 7
7. Sistema de Caché y Resiliencia Offline	Pág. 8
8. Marco de Seguridad y Gestión de Credenciales	Pág. 9
9. Guía de Compilación, Pruebas y Despliegue	Pág. 10

1. Resumen de Arquitectura y Patrón BFF

Pantry Guard está estructurado bajo el patrón arquitectónico **Backend-for-Frontend (BFF)**. Esta decisión de ingeniería responde directamente a los requerimientos de seguridad para modelos de lenguaje en dispositivos móviles: *bajo ninguna circunstancia las llaves criptográficas de APIs comerciales (Google Gemini API) deben residir en el binario del cliente móvil (APK/PWA).*

Principio de Aislamiento BFF: El cliente móvil jamás interactúa directamente con los servidores de Google GenAI ni con bases de datos remotas sin intermediación. Toda petición se envía a través de endpoints locales protegidos del servidor Express, el cual actúa como guardián de validación de esquemas, sanitización de datos y limitador de tasa de consumo.

El flujo de datos se resume en la siguiente topología de comunicación:



```
▼ |  
[ Servidor Express (server.ts) en Render ]  
├─ 1. Middleware de CORS y Headers de Seguridad (HSTS)  
├─ 2. Capa de Rate-Limiting por IP/Usuario (anti-EDoS)  
├─ 3. Memoria Caché de Consultas (MEM_CACHE)  
├─ 4. Motor de Recetas Locales de Emergencia (generateLocalRecipes)  
└─ 5. Conector Seguro con Google GenAI SDK (process.env.GEMINI_API_KEY)  
    │  
    ├─ Google Gemini API (Inferencia multimodal estructurada)  
    └─ Open Food Facts API (Resolución global de códigos EAN/UPC)
```

2. Stack Tecnológico y Matriz de Dependencias

El sistema combina herramientas de última generación en el ecosistema JavaScript/TypeScript tanto para el renderizado visual de alta tasa de cuadros (60 FPS) como para la compilación optimizada en producción:

Capa	Tecnología	Versión	Rol y Justificación en el Proyecto
Frontend	React	19.0.1	Renderizado reactivo con gestión de estado atómica y concurrencia.
Empaquetador	Vite	6.2.3	Compilador ultrarrápido con soporte nativo de ES Modules y HMR.
Tipado	TypeScript	5.8.2	Garantía de tipado estricto en interfaces de productos y respuestas de IA.
Estilos UI	Tailwind CSS	4.1.14	Framework utility-first con variantes dinámicas de modo oscuro y clases seguras.
Backend	Express	4.21.2	Servidor HTTP minimalista para orquestación de APIs REST y serving de la SPA.
Compilación CJS	esbuild	0.25.0	Empaquetador ultraeficiente de <code>server.ts</code> a <code>dist/server.cjs</code> en menos de 20ms.
IA Generativa	@google/genai	2.4.0	SDK oficial de Google Cloud para modelos Gemini 2.5 Flash y Gemini 3.8.
Cámara / Barcode	Html5Qrcode + ZXing	2.3.8 / 0.23.0	Decodificación de video en vivo asistida por <code>window.BarcodeDetector</code> .

3. Estructura de Directorios del Repositorio

El código fuente se organiza siguiendo el principio de separación de responsabilidades:

```
Pantry-Guard/  
├─ src/  
│   └─ components/                # Componentes modulares de interfaz de usuario  
│       └─ AIInsightsView.tsx      # Módulo Chef IA, filtros de categorías y recetas  
│       └─ BarcodeScannerModal.tsx # Escáner de vídeo en vivo, retícula y hardware detector  
│       └─ ReceiptScanModal.tsx    # Escáner OCR de tickets de compra con Gemini Vision  
│       └─ ProductModal.tsx        # Formulario de alta, edición y ajuste de cantidades
```

```

├── ProductCard.tsx      # Tarjeta individual con cálculo de semáforo de días
├── PantryListView.tsx   # Vista general con búsqueda en tiempo real y agrupaciones
├── ShoppingListView.tsx # Lista dinámica con checkboxes interactivos
├── TopNavBar.tsx       # Barra superior con selectores de idioma (ES/EN) y tema
├── data/
│   └── initialData.ts   # Semilla inicial de productos y catálogo local offline
├── utils/
│   ├── barcodeService.ts # Consultas Open Food Facts, MEM_CACHE y caducidades
│   ├── imageBarcodeDecoder.ts # Decodificador cliente multinivel (Hardware/ZXing)
│   ├── i18n.ts          # Diccionario completo de internacionalización bilingüe
│   └── storage.ts        # Capa de abstracción para persistencia en LocalStorage
├── types.ts            # Contratos de datos, enums y tipos TypeScript
├── App.tsx             # Orquestador del estado raíz, modales y navegación
├── main.tsx            # Punto de entrada y montaje en el DOM
├── index.css           # Tailwind base, reglas de cámara y animaciones
├── server.ts           # Backend Express, endpoints /api/ai/* y fallback local
├── vite.config.ts      # Configuración de compilación Vite y plugins
├── package.json        # Manifiesto de dependencias y scripts de ejecución
├── capacitor.config.json # Configuración para empaquetado nativo en Android Studio
└── .gitignore          # Exclusiones de Git (garantiza que .env jamás se suba)

```

4. Modelado de Datos e Interfaces TypeScript

Toda la información del sistema se modela estrictamente a través de tipos e interfaces en `src/types.ts`:

Entidad Principal: Producto (Product)

```
export type ProductCategory = 'lacteos' | 'proteinas' | 'verduras' | 'frutas' | 'granos' | 'bebidas';
export type StorageLocation = 'Refrigerador' | 'Alacena' | 'Frutero' | 'Congelador';
export type ProductStatus = 'fresh' | 'expiring' | 'expired';

export interface Product {
  id: string;
  name: string;
  category: ProductCategory;
  quantity: number;
  unit: 'unidades' | 'kg' | 'g' | 'L' | 'ml' | 'lb' | 'oz';
  location: StorageLocation;
  expiryDate: string; // Formato estricto ISO 'YYYY-MM-DD'
  barcode?: string;
  imageUrl?: string;
  brand?: string;
  nutriScore?: string;
  notes?: string;
}
```

Entidad: Receta Generada por IA (AIRecipe)

```
export interface AIRecipe {
  id: string;
  title: string;
  mealType: 'desayuno' | 'almuerzo_cena' | 'express' | 'postre_snack';
  cookTimeMinutes: number;
  difficulty: 'fácil' | 'media' | 'avanzada';
  pantryIngredientsUsed: Array<{
    name: string;
    quantity: string;
    isExpiringSoon: boolean;
  }>;
  extraIngredientsNeeded: Array<{ name: string; isOptional: boolean }>;
  steps: string[];
  zeroWasteTip: string;
}
```

5. Especificación de Endpoints y Servicios API

El backend expone rutas REST unificadas bajo el prefijo `/api/ai`:

1. POST `/api/ai/recipes`

- **Propósito:** Generación de recetas cero desperdicio adaptadas al inventario vigente.
- **Control de Validación:** Se usa `responseSchema` con tipos `Type.OBJECT` y `Type.ARRAY` en el SDK de Gemini.
- **Manejo de Errores:** Si el LLM devuelve un error de cuota (429) o saturación (503), el catch invoca `generateLocalRecipes()`, entregando recetas deterministas en menos de 10ms sin interrumpir la experiencia de usuario.

2. POST `/api/ai/scan-receipt`

- **Propósito:** Reconocimiento óptico de caracteres (OCR) y análisis de recibos de supermercado.
- **Payload:** Objeto JSON con `imageBase64` codificado en Base64 JPEG y tamaño máximo de 2 MB.
- **Salida:** Array de artículos detectados con cantidades, categorías y caducidades asignadas según reglas alimentarias.

6. Mecanismos de Visión Computacional y Escaneo 1D

El módulo `BarcodeScannerModal.tsx` implementa una arquitectura híbrida de escaneo en tiempo real orientada a dispositivos móviles:

- **Hardware Acceleration** (`window.BarcodeDetector`): En navegadores modernos (Android Chrome e iOS 17+), se ejecuta un bucle de inspección a 25 FPS (cada 40ms) conectado directamente a la GPU del dispositivo. Esto permite detectar códigos de barras EAN-13 en menos de 15 ms sin bloquear el hilo principal de JavaScript.
- **Enfoque Continuo por Hardware**: Al iniciarse el video, se negocian las capacidades de la cámara con `applyConstraints({ advanced: [{ focusMode: 'continuous' }] })` para garantizar nitidez macro a 10–15 cm de distancia.
- **Visor Panorámico Único**: Se ocultan los elementos predeterminados de la librería externa con CSS (`#qr-shaded-region { display: none !important; }`) para mostrar un único marco con relación de aspecto 16:10 y línea láser animada.

7. Sistema de Caché y Resiliencia Offline

Para mitigar la latencia de servicios externos (Open Food Facts y Gemini) y optimizar costos operativos, se implementa una caché en memoria:

```
// Implementación en barcodeService.ts
const MEM_CACHE: Record = {};

export async function lookupBarcode(barcode: string): Promise {
  const cleanCode = barcode.trim();

  // 0. Consulta instantánea en memoria (0ms)
  if (MEM_CACHE[cleanCode]) return MEM_CACHE[cleanCode];

  // 1. Consulta en memoria de productos aprendidos (LocalStorage)
  // 2. Consulta en catálogo estático precargado (initialData.ts)
  // 3. Consulta en vivo a Open Food Facts con timeout rápido (2.5 segundos)
}
```

8. Marco de Seguridad y Gestión de Credenciales

- **Protección de Secretos**: La variable `GEMINI_API_KEY` reside exclusivamente en el entorno del servidor Node.js y en los secretos encriptados de Render. El archivo `.gitignore` contiene la

directiva `.env*` para evitar cualquier commit accidental.

- **Prevención de Inyección de Prompts:** Toda entrada proveniente de tickets u Open Food Facts se procesa como contenido de datos plano dentro de etiquetas semánticas y nunca se concatena como instrucción de sistema.
- **Salidas Controladas:** Cero uso de `eval()` o inyección de scripts dinámicos en la interfaz de React.

9. Guía de Compilación, Pruebas y Despliegue

Comandos del Proyecto:

Comando	Propósito
<code>npm run dev</code>	Inicia el entorno de desarrollo local con recarga rápida (HMR).
<code>npm run build</code>	Compila los bundles de Vite y empaqueta el servidor con esbuild en <code>dist/server.cjs</code> .
<code>npm start</code>	Inicia el servidor de producción Express sirviendo los assets estáticos.
<code>npm run lint</code>	Ejecuta la verificación de tipos de TypeScript sin emitir archivos (<code>tsc --noEmit</code>).

Despliegue Continuo en Render:

1. Conectar el repositorio `AAMC98/PantryGuard` en Render.com como **Web Service**.
2. Configurar **Build Command**: `npm install && npm run build`
3. Configurar **Start Command**: `npm start`
4. Agregar en **Environment Variables**: `GEMINI_API_KEY` con la clave de Google AI Studio.